Explaining the Spread: Decomposing an FX Quote in kdb+/q

Summary

A pricing engine's quoted spread is never just one number — it's the sum of several named adjustments stacked on top of a reference level: a base markup, a client-tier skew, a volatility buffer, quote-stability smoothing, a fallback component, and a directional-signal adjustment. In the context of this article and the code:

- **Composition** asks the simple direction: given the named components, what's the total quoted spread? A one-line row-wise sum.
- **Decomposition and aggregation** ask the harder question: once you have thousands of quotes a day across symbols, aggression levels, and market regimes, how much of the spread is coming from *which* component, on average, and does that hold up when you slice by time instead of by tag? Get the weighting wrong and every rollup lies.
- **Reconciliation** asks a third question: is what the model quoted actually consistent with an independent reference — richer, cheaper, in line?

This is a different shape of problem to markout / market impact. There, the components are hidden — you observe a single noisy price curve after a trade and have to *infer* a temporary/permanent split from it. Here, the components are already columns in the row the moment the quote is generated; nothing needs to be estimated. The engineering problem is entirely downstream: aggregating an additive decomposition without breaking the weighting, and checking the result against something outside the model.

Repo

- The code discussed in this article is at: **https://github.com/SpencerFX/q-link**
- To load the functions, synthetic data, and explore interactively:

```
q ./scripts/initSpread.q
```

- To run the test suite (hard assertions, ground-truth checks included):

```
q ./test/testSpread.q
```

The component model

A quote is modelled as seven named components summing to a total:

```
.spread.componentCols:`refSprd`baseSprd`clientSprd`volSprd`smoothSprd`fallbackSprd`alphaSprd;

.spread.compose:{[tab]
  update totalSprd:sum value flip .spread.componentCols#tab from tab
 };
```

Component	What it represents
`refSprd`	reference/baseline spread before any adjustment
`baseSprd`	core pricing-engine markup
`clientSprd`	client-tier/relationship skew
`volSprd`	volatility risk buffer — widens under elevated vol
`smoothSprd`	quote-stability smoothing — dampens jumps between quotes
`fallbackSprd`	supplemental buffer, used when other inputs are thin
`alphaSprd`	directional-signal adjustment

```
q)meta scenario`quotes
c           | t f a
------------| -----
time        | p   s
sym         | s
aggression  | s
marketStatus| s
weight      | f
refSprd     | f
baseSprd    | f
clientSprd  | f
volSprd     | f
smoothSprd  | f
fallbackSprd| f
alphaSprd   | f
totalSprd   | f
```

`.spread.decompose` melts that into one row per (quote, component) — the shape a stacked-bar or attribution view wants — and `.spread.waterfall` appends a running cumulative column per component, so `cum_alphaSprd == totalSprd` on every row by construction. Both are exact, not fitted: no residual, no goodness-of-fit statistic, because there's nothing being estimated.

One weighting rule, three entry points

The part worth being careful about is aggregation. A single quote's spread means nothing on its own — the number that matters is the size-weighted average across many quotes, and that weighting has to be applied *consistently* whether you're rolling up by regime, by time, or by nothing at all. Rather than writing that three times, one private helper builds the aggregate-column spec and every public rollup threads through it:

```
.spread.priv.wavgAggCols:{[wCols]
  aCols:enlist[`weight]!enlist(sum;`weight);
  aCols,raze {enlist[x]!enlist(wavg;`weight;x)} each wCols
 };

.spread.wavgBy:{[tab;keyCols]
  t:$[`totalSprd in cols tab;tab;.spread.compose tab];
  wCols:.spread.componentCols,`totalSprd;
  ?[t;();keyCols!keyCols;.spread.priv.wavgAggCols wCols]
 };
```

`.spread.byTime` and `.spread.byRegime` are both a handful of lines on top of the same helper — `byTime` swaps in a time-bucket parse-tree as the group-by key, `byRegime` just calls `wavgBy` with `` `aggression`marketStatus `` fixed as the keys. One weighting convention, three ways to slice it.

On data

To check the aggregation and reconciliation logic against a known answer rather than plausible-looking output, `data/spreadGenerator.q` builds a synthetic session with three effects injected in advance:

- **A market-status regime shift.** The first half of the session is tagged `normal`, the second half `stressed`, and `volSprd` is multiplied by a known factor (**4.0x**) for every stressed quote. Nothing else is touched.
- **An aggression tightening.** `baseSprd` and `clientSprd` scale by a known per-aggression-level multiplier (`low`=1.0, `medium`=0.7, `high`=0.4) — more aggressive pricing quotes tighter.
- **An independent benchmark series** built from the model's own `totalSprd` minus a known constant offset (**0.05** price units, i.e. 500 on the `1e4*` bps convention `.spread.vsReference` uses) plus noise — standing in for a rate the model didn't produce itself, for testing reconciliation.

```
stressFactor:?[marketStatus=`stressed;.spreadSynth.config.stressVolMult;1f];
volSprd:0.1*baseLevel*stressFactor*noise[n];
...
benchmark:update benchmarkSprd:totalSprd-richness+0.01*.spreadSynth.priv.randNorm[n]
  from select time,sym,totalSprd from quotes;
```

Note: this generator isn't a model of how a real pricing engine actually sets a spread — it's a controlled way to know the right answer in advance, the same role GBM plays in the markout/impact piece's synthetic rate series.

Interpreting the results

`scripts/initSpread.q` builds a 6,000-quote synthetic session (3 symbols, 3 aggression levels, two regimes), runs it through `.spread.byRegime` and `.spread.vsReference`, and leaves `recovery` in the workspace — the same check `test/testSpread.q` asserts on non-interactively:

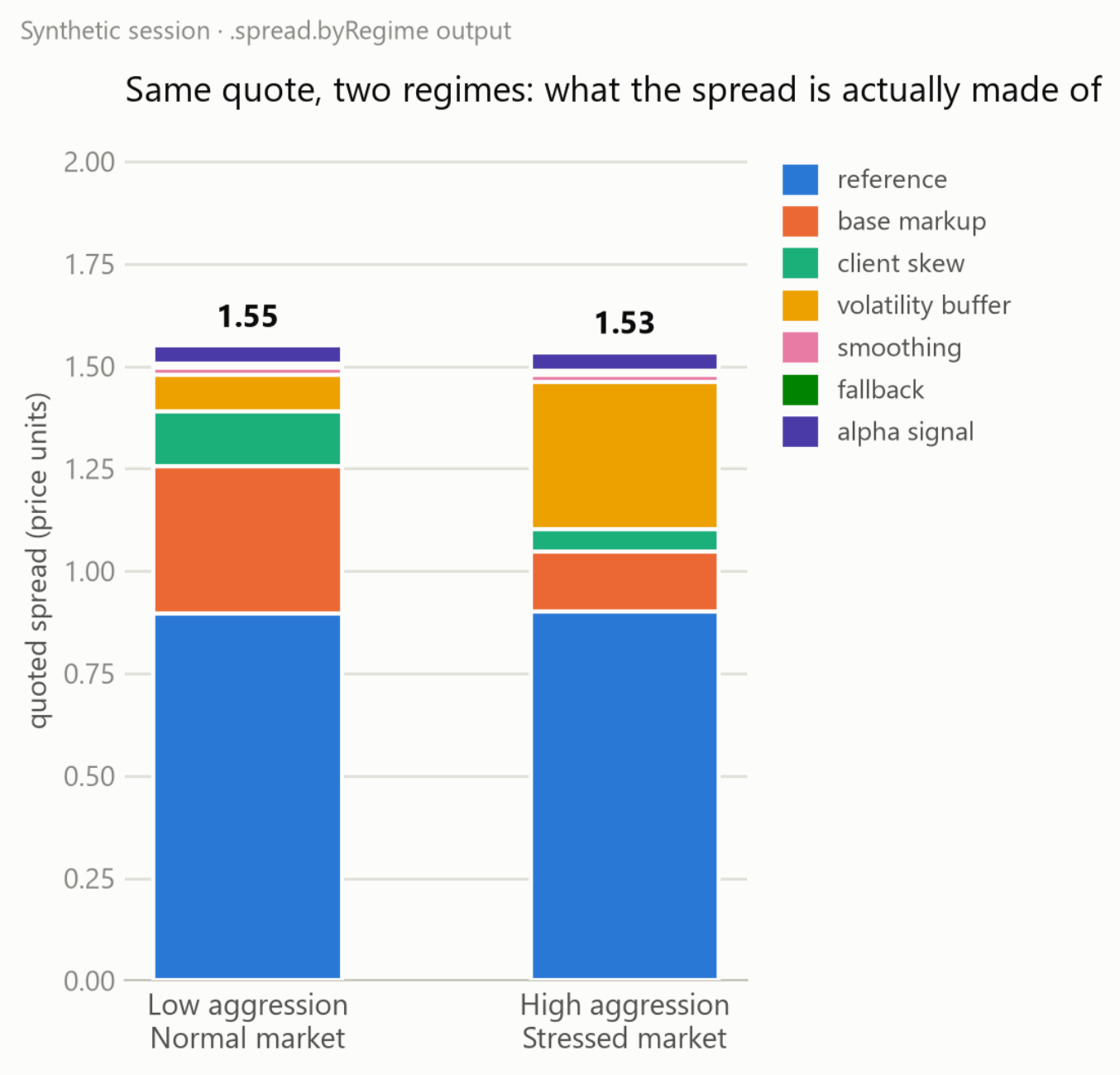
```
q)recovery
check         expected recovered relErrPct  pass
-----------------------------------------------
stressVolMult 4        4.004076  0.101891   1
richnessBps   500      500.2515  0.05029823 1
```

- **stressVolMult** — grouping all 6,000 quotes by `marketStatus` alone and taking the ratio of average `volSprd` (stressed ÷ normal) recovers the injected 4.0x multiplier to within ~0.1%.
- **richnessBps** — joining the model's composed `totalSprd` against the independent benchmark series on `` `time`sym `` via `.spread.vsReference` and averaging the recovered `richnessBps` recovers the injected 500-unit richness to within ~0.05%.

The aggression effect isn't in the automated check above, but it's directly visible in `.spread.byRegime`'s output — comparing `low` against `high` aggression *within the same* `normal` market status:

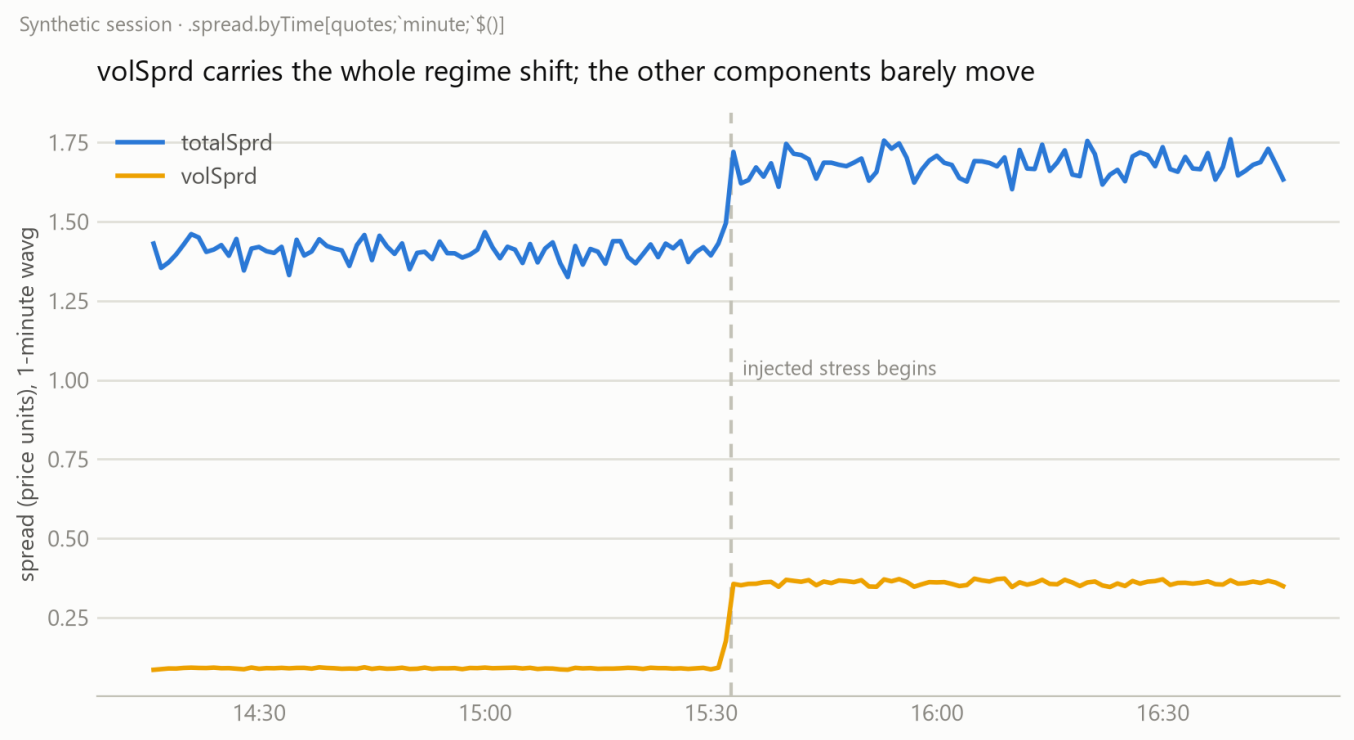
```
aggression marketStatus  baseSprd   clientSprd
low        normal        0.3585052  0.1343631
high       normal        0.1438458  0.05404622
```

`0.1438458 / 0.3585052 ≈ 0.401` and `0.05404622 / 0.1343631 ≈ 0.402` — both land on the injected `aggressionMult[`high]` of 0.4, recovered independently for two different components.



Same quote, priced two ways

That chart picks two realistic composite scenarios rather than isolating one variable — a calm, low-aggression quote versus an aggressive quote issued into a stressed market — and it's worth noticing what the totals do: **1.55 vs. 1.53, almost unchanged.** The tighter base markup and client skew from aggressive pricing very nearly cancel the wider volatility buffer from the stress regime. A dashboard that only shows `totalSprd` would report these two quotes as practically identical; the decomposition shows they got there by two completely different routes. That gap — same total, different composition — is the entire reason to keep the components around instead of collapsing to one number at write time.



volSprd carries the regime shift

The second chart rolls the same session up by `.spread.byTime[quotes;`minute;`$()]` instead of by regime tag — a completely independent aggregation path — and recovers the same step: `volSprd` jumps at the injected transition, `totalSprd` follows it, and every other component stays flat. Two unrelated rollups agreeing on the same signal is itself a form of validation.

Reconciliation vs. an outside reference

`.spread.vsReference` joins the model's composed total against any independently sourced spread series and reports richness in both bps and pct:

```
.spread.vsReference:{[modelTab;refTab;keyCols;refCol]
  m:$[`totalSprd in cols modelTab;modelTab;.spread.compose modelTab];
  mSel:keyCols xkey ?[m;();0b;(keyCols!keyCols),enlist[`modelSprd]!enlist`totalSprd];
  rSel:keyCols xkey ?[refTab;();0b;(keyCols!keyCols),enlist[`refSprd]!enlist refCol];
  res:0!mSel,'rSel;
  update richnessBps:1e4*modelSprd-refSprd, richnessPct:100*(modelSprd-refSprd)%refSprd from re
s
 };
```

The obvious use is a pricing-desk sanity check — is what we're quoting consistent with a competitor feed, a prior model version, or an internal benchmark rate — but it's the same shape regardless of what `refTab` actually is: any two independently produced spread series, reconciled on shared keys.

Conclusions

Spread decomposition and market impact both live under the same broad heading — pricing/post-trade analytics — and they're close to opposite problems. Market impact starts with one noisy number (price after the fact) and

has to recover a hidden temporary/permanent structure from it, which is why that piece needed a parametric fit and a goodness-of-fit statistic to know if the recovery was any good. Spread decomposition starts with the structure already given — every component is a column the pricing engine already computed — so there's nothing to estimate. The entire job is downstream: aggregate an additive decomposition without breaking the weighting no matter which dimension you slice by, and check the result against something the model didn't produce itself.

Both pieces lean on the same discipline to know the code is actually right rather than just plausible: build a synthetic scenario with a known answer baked in, and require the functions to reproduce that number, not merely something in the right ballpark.